

The parameters passed to *pcap_compile* are as follows:

- handler – This is the session handler obtained earlier.
- filter – This is the location where you store the compiled filter.
- filt – This is the string that represents your filter. The *tcpdump* man pages document the syntax of these filters well.
- opt – Specifies whether the compiled filter should be optimised or not.
- netmask – Specifies the network mask that your filter applies to.

Once the filter has been compiled, you must now apply it. Use the following function to do so:

```
int set_filter(pcap_t *handler, struct bpf_program *filter)
```

...where handler is the session handler, and filter is the location where your compiled filter resides. This is straightforward.

The following code documents how filters must be used:

```
#include<stdio.h>
#include<pcap.h>
int main()
{
    char *dev,errbuf[PCAP_ERRBUF_SIZE];
    dev=pcap_lookupdev(errbuf);
    pcap_t *handle;
    handle=pcap_open_live(dev,BUFSIZ,1,0,errbuf);
    struct bpf_program filter;
    char filter_app[]="port 80";
    bpf_u_int32 mask,net;
    pcap_lookupnet(dev,&net,&mask,errbuf);
    pcap_compile(handle,&filter,filter_app,0,net);
    pcap_setfilter(handle,&filter);
    return 0;
}
```

In the above code, the function *pcap_lookupnet* is used to identify the network details, like the IP and network mask, given the device. Notice that the variable *filter_app* stores the string "port 80", which is your filter string, which specifies that only traffic on port 80 should be captured. Port 80 is the TCP port. The syntax for other filter commands is documented in the *tcpdump* man pages.

Actual sniffing

Now that you have your filters, session and device set up, you need to actually start capturing packets, which you can do using the following function:

```
u_char * pcap_next(pcap_t *handler,struct pcap_pkthdr *h)
```

...where 'handler' is the session handler mentioned earlier and 'h' is a structure that will store a pointer to information about

the packet to be captured. This includes the time it was sniffed, the packet length, and the length of the specific portion of the packet captured (in case it is fragmented). The function returns a pointer of type *u_char* to a packet that is described by 'h'.

The code to use the *pcap_next* is given below:

```
#include<stdio.h>
#include<pcap.h>
int main()
{
    char dev[]="wlan0",errbuf[PCAP_ERRBUF_SIZE];
    pcap_t *handle;
    handle=pcap_open_live(dev,BUFSIZ,1,0,errbuf);
    struct bpf_program filter;
    char filter_app[]="port 80";
    bpf_u_int32 mask,net;
    pcap_lookupnet(dev,&net,&mask,errbuf);
    pcap_compile(handle,&filter,filter_app,0,net);
    pcap_setfilter(handle,&filter);
    struct pcap_pkthdr header;
    const u_char *packet;
    packet=pcap_next(handle,&header);
    printf("%d \n",header.len);
    pcap_close(handle);
    return 0;
}
```

In the above code, you captured the packet and printed the length of the packet captured. Also note that, in the above code, the device string has been set to use the *wlan0* connection to listen to packets on the wireless device. To see the list of available devices on your Linux system, you can use the *ifconfig* command at the terminal, and set your device accordingly.

In practice, very few sniffers use *pcap_next* directly. They usually use *pcap_loop* or *pcap_dispatch* to capture multiple packets. They use *callback* functions. If you have worked with Javascript, you might be familiar with a *callback* function. It can be understood as a function that is called, every time an event occurs. The functions *pcap_loop* and *pcap_dispatch* call the *callback*, every time a packet is captured. The signature for *pcap_loop* is given below:

```
int pcap_loop(pcap_t *handler,int cnt,pcap_handler
callback,u_char *user)
```

The parameters for *pcap_loop* are defined below:

- handler – This is the session handler.
- cnt – This is the number of packets which should be captured by *pcap_loop*.
- callback – This is the callback function specified only by identifier, without the parentheses.
- user – These are some additional arguments that we as users might want to pass to the *callback* function.